

National Institute of Technology Karnataka, Surathkal

Department of Computer Science and Engineering

Lab Assignment – 3

Language Allowed: C++

Course: Data Structures and Algorithms

Academic Year: 2026-27

Due Date: 06/09/2026, 12 PM

Section: S1 and S2

NOTE : Implement all questions.

Q1. NITK Hostel Common Service Counter: Unified Queue System

The NITK hostel administration has set up **one common service counter** near the warden's office that handles three different kinds of student requests every evening:

- (a) **Mess Complaint:** a student reports a food-quality issue. Takes a fixed base handling time, plus 2 extra minutes if marked **SEVERE**.
- (b) **Maintenance Request:** a student reports a broken fan/light/plumbing issue. Takes a fixed base handling time, plus 5 extra minutes if the issue is **ELECTRICAL** (needs a certified electrician to be called).
- (c) **Leave Application:** a student requests permission to leave campus for the weekend. Takes a fixed base handling time, plus 3 extra minutes if the destination is **OUTOFSTATE** (needs extra verification).

All three request types share common information a token id, the student's name, and a base handling time but each computes its **actual handling time** differently, and each produces a **different printed receipt format** when served. The clerk at the counter does not want to know or care what kind of request it is; she just pulls the next token from the queue and calls a generic `serve()` on it, and the correct behaviour must happen automatically depending on what that request actually is.

Requests arrive throughout the evening in any order and must be served strictly **FIFO** one common queue for all request types, not separate queues.

Task

Given a sequence of incoming requests (each request specifying its type and type-specific details), simulate the queue and, for each request, print its receipt when it is served, and finally print the total number of requests served and the total time the counter was active.

Input Format

N
type[1] token[1] name[1] base_time[1] extra_field[1]

```
type[2] token[2] name[2] base_time[2] extra_field[2]
...
type[N] token[N] name[N] base_time[N] extra_field[N]
```

- `type` $\in \{\text{MESS, MAINT, LEAVE}\}$
- `extra_field` depends on `type`:
 - `MESS` $\rightarrow$ `SEVERE` or `NORMAL`
 - `MAINT` $\rightarrow$ `ELECTRICAL` or `OTHER`
 - `LEAVE` $\rightarrow$ `OUTOFSTATE` or `INSTATE`

Sample Input

```
5
MESS 1 Sahaj 5 SEVERE
MAINT 2 Raunak 8 ELECTRICAL
LEAVE 3 Subham 4 OUTOFSTATE
MESS 4 Sunny 5 NORMAL
LEAVE 5 Sanjeev 4 INSTATE
```

Sample Output

```
[MESS RECEIPT] Token 1 | Sahaj | Handling time: 7 min | SEVERE
[MAINT RECEIPT] Token 2 | Raunak | Handling time: 13 min | ELECTRICAL
[LEAVE RECEIPT] Token 3 | Subham | Handling time: 7 min | OUTOFSTATE
[MESS RECEIPT] Token 4 | Sunny | Handling time: 5 min | NORMAL
[LEAVE RECEIPT] Token 5 | Sanjeev | Handling time: 4 min | INSTATE
Total requests served: 5
Total counter active time: 36 minutes
```

Design Requirements (read carefully this is the core of the assignment)

This problem must be solved by first designing a Queue ADT.

1. Abstraction + Encapsulation the Request hierarchy

- Define an **abstract base class** `Request` that captures what is common to *any* request: token id, student name, base handling time (as **protected** members **not** public), and:
 - a **pure virtual function** `int getHandlingTime()` computes the actual handling time, differently for each type.
 - a **pure virtual function** `void serve()` prints the receipt in the type-specific format.
- You may also include a normal (non-virtual) member function common to all, e.g. `int getTokenId() const`, to show you understand that not *everything* in a base class needs to be virtual.

- Because **Request** has pure virtual functions, it is abstract you should never be able to create a **Request** object directly, only through its derived classes. Your write-up (as a comment) should explicitly state why this class is abstract and what “abstraction” means in this context.
- All data members must be **private** or **protected**, accessed only via constructors and member functions no public data members. This is your **encapsulation** requirement.

2. Inheritance three derived classes

- **MessComplaint**, **MaintenanceRequest**, and **LeaveApplication** must each **publicly inherit** from **Request**.
- Each derived class adds only the extra field it specifically needs (**severe** flag, **electrical** flag, **outOfState** flag respectively) and must call the base class constructor properly (via constructor initializer list) to set up the common fields.

3. Polymorphism - the key test

- The **Queue ADT must store Request*** (base class pointers), regardless of which derived object they actually point to.
- When you dequeue a request and call `->serve()` or `->getHandlingTime()` on the base pointer, the **correct derived class version must run automatically** this is runtime polymorphism via virtual functions, and it is what allows the clerk (your `main()` simulation loop) to serve any request type without an `if/else` chain checking the type.
- Explicitly avoid writing code like `if (type == "MESS") ... else if (type == "MAINT") ...` at the point of *serving* this defeats the purpose of polymorphism. Such type-checking is allowed only once, at the point where you *construct* the correct derived object while reading input (since input parsing has to know what to construct).

4. Queue ADT mandatory

- Design the **Queue ADT** first, in a written note before coding: state its data members, and its operations (**enqueue**, **dequeue**, **isEmpty**, **isFull**, **peek**, **size**).
- The queue stores **Request*** (pointers to the abstract base class) this is what makes polymorphism through the queue possible.
- Implement the queue **from scratch using a plain array or a self-built linked list** no `std::queue`, `std::vector`, `std::list`, `std::deque`, or any STL container/adaptor.
- Since the queue stores **pointers**, not objects, be prepared to explain (in a comment) why storing pointers here is necessary what would go wrong if you tried to store **Request** objects *by value* in the array instead?

Additional Implementation Instructions

- (a) **C++ only** use `cin/cout/ <iostream>` throughout. No `scanf/printf` or other C-style I/O. A program using C-style I/O will not be accepted as a valid C++ submission.

- (b) Create each **Request** derived object with **new** while parsing input. Raw pointers with **new** are expected here (do *not* use `std::shared_ptr`/`std::unique_ptr` they would hide the pointer mechanics this assignment is testing). Remember to **delete** each request after it has been served, to avoid memory leaks this will be checked.
- (c) Structure your submission with a short header comment covering:
 - Your Queue ADT design (data members + operations).
 - One or two lines each explaining *where* abstraction, encapsulation, inheritance, and polymorphism appear in your specific solution.
- (d) **main()** should only: (a) parse input and construct the correct derived object per request, (b) enqueue it (as a **Request***), (c) in a loop, dequeue and call **serve()**/**getHandlingTime()** polymorphically, (d) accumulate and print the final totals. **main()** should contain **no** per-type receipt-printing logic itself that logic belongs entirely inside each derived class's **serve()**.

Q2. DNA Palindrome Using a Custom Deque

A biotechnology firm is analyzing genetic sequences to identify palindromic DNA sequences. A DNA sequence is called a palindrome if it reads the same forwards and backwards.

The machine first loads the DNA sequence character by character into a double-ended queue (Deque). To verify whether the sequence is a palindrome, the machine simultaneously removes one character from the front and one character from the back of the deque and compares them.

- If the two characters are different, the sequence is not a palindrome.
- If they are the same, the process continues.
- The process stops when the deque becomes empty or contains only one character.

Mandatory Object-Oriented Requirements:

- Implement the solution using Object-Oriented Programming concepts.
- Create a `Node` class/structure to represent each character in the deque.
- Create a user-defined `Deque` class from scratch.
- The Deque must be implemented using pointers and a linked list.
- The Deque class must maintain appropriate private data members such as:
 - Pointer to the front node.
 - Pointer to the rear node.
 - Number of elements in the deque.
- Implement public member functions such as:
 - `pushFront()` or `pushBack()` to insert characters.
 - `popFront()` to remove a character from the front.
 - `popBack()` to remove a character from the rear.
 - `isEmpty()` to check whether the deque is empty.
 - `size()` to return the number of elements.
- Use a constructor to initialize the deque.
- Use a destructor to release all dynamically allocated memory.
- Keep the deque's data members private and access them only through public member functions.
- **Do not use any built-in STL containers or data structures** such as `vector`, `deque`, `list`, `string`, or `arrays`.
- The palindrome-checking logic must operate using the custom `Deque` class rather than directly checking the input sequence.

Input Format:

The first line contains an integer N , representing the length of the DNA sequence.

The second line contains a DNA sequence of N uppercase characters. Each character is one of: A, C, G, T.

Output Format:

Print YES if the DNA sequence is a palindrome.

Otherwise, print NO.

Constraints:

- $1 \leq N \leq 10^5$
- The sequence contains only uppercase characters A, C, G, and T.
- The algorithm must run in $O(N)$ time.
- The deque must use dynamic memory and pointers.
- No STL containers or arrays may be used.

Example 1:

Input:

```
6
ACGTCA
```

Output:

```
NO
```

Example 2:

Input:

```
7
ACGCGCA
```

Output:

```
YES
```

Q3. Postfix Expression Evaluation – Billing Engine

Scenario:

A billing and invoicing application used by sales executives needs a calculation engine. When an executive types an expression such as $(45 + 30) * 2 - 15/3$ to quickly compute a quote, the app internally converts it into postfix notation and evaluates it using a stack, since postfix evaluation avoids all ambiguity around operator precedence and parentheses.

The backend team wants the operator logic to be easily extensible – adding a new operator later should mean writing **one new class**, without ever touching the core evaluation loop. So instead of checking the operator symbol at the point of computation, each operator is represented as its own class, and the evaluation loop calls a single, generic function on it – the correct arithmetic must happen automatically depending on which operator it actually is.

Task:

Given a sequence of postfix expressions, evaluate each one using an operand Stack ADT together with the **Operator** class hierarchy described below, printing either the result or the appropriate error message for each expression.

Design Requirements (read carefully – this is the core of the assignment)

This problem must be solved using abstraction, encapsulation, inheritance, and polymorphism together with a Stack ADT.

1. Abstraction + Encapsulation – the Operator hierarchy

- Define an abstract base class **Operator** with a **protected** member for its symbol (set via constructor), and:
 - a pure virtual `long long apply(long long a, long long b) const` – computes the result of applying this operator to two operands.
 - a pure virtual `bool isValid(long long a, long long b) const` – returns whether the operation is well-defined for these operands (e.g., **false** when the divisor is zero for `/` or `%`; always **true** for `+`, `-`, `*`).
- Include one non-virtual member function common to all, e.g. `char getSymbol() const`, to show you understand that not everything in a base class needs to be virtual.
- Because **Operator** has pure virtual functions, it is abstract – you should never be able to create an **Operator** object directly, only through its derived classes. Your header comment must explicitly state why this class is abstract and what “abstraction” means in this context.
- All data members must be **private** or **protected**, accessed only via constructors and member functions – no public data members. This is your encapsulation requirement.

2. Inheritance – five derived classes

- **AddOperator**, **SubtractOperator**, **MultiplyOperator**, **DivideOperator**, and **ModuloOperator** must each publicly inherit from **Operator**.
- Each must call the base class constructor (via constructor initializer list) to set up its symbol.
- Each implements `apply()` with its specific arithmetic, and `isValid()` – only **DivideOperator** and **ModuloOperator** need to actually check for a zero divisor; the rest simply return **true**.

3. Polymorphism – the key test

- When your evaluation loop needs to apply an operator, it must already hold the correct `Operator*` (base class pointer) for that token.
- Calling `op->isValid(a, b)` and `op->apply(a, b)` on that base pointer must automatically run the correct derived class's version – this is runtime polymorphism via virtual functions, and it is what allows the evaluation loop to apply any operator without an if/else chain checking the symbol.
- Explicitly avoid writing code like `if (symbol == '+') ... else if (symbol == '-') ...` at the point of applying the operator – this defeats the purpose of polymorphism. Such type-checking is allowed only once: inside a small factory function used while scanning tokens, to decide which derived `Operator` object to construct.

4. Stack ADT – mandatory

- Design the Stack ADT first, in a written note before coding: state its data members, and the operations you have decided your evaluation logic needs.
- The stack stores operand values (`long long`), implemented from scratch using a plain array or a self-built linked list – no `std::stack`, `std::vector`, `std::list`, `std::deque`, or any STL container/adaptor.

Additional Implementation Instructions

1. **C++ only** – use `cin/cout/<iostream>` throughout. No `scanf/printf` or other C-style I/O. A program using C-style I/O will not be accepted as a valid C++ submission.
2. Construct the correct `Operator` derived object with `new` inside the factory function, when an operator token is scanned. Raw pointers with `new` are expected here (do not use `std::shared_ptr/std::weak_ptr` – they would hide the pointer mechanics this assignment is testing). After applying it to the popped operands, `delete` it immediately, before moving to the next token – to avoid memory leaks (this will be checked).
3. Structure your submission with a short header comment covering:
 - Your Stack ADT design (data members + operations).
 - One or two lines each explaining where abstraction, encapsulation, inheritance, and polymorphism appear in your specific solution.
 - Why storing an `Operator*` (pointer to the abstract base) is necessary – what would go wrong if you tried to store `Operator` objects by value instead?
4. Your evaluation loop should only: tokenize each expression, push number tokens onto the stack, and for operator tokens – use the factory to construct the correct `Operator*`, pop two operands, check `isValid()`, call `apply()` polymorphically, delete the operator object, and push the result. No per-operator arithmetic should appear anywhere outside each derived class's `apply()`.

Expected Input:

The first line contains an integer q , the number of postfix expressions to evaluate. Each of the next q lines contains one postfix expression, with tokens (operands and operators) separated by single spaces.

Expected Output:

For each expression, print a single line containing either the integer result of evaluating the expression, or one of the following error messages, if applicable: `Incomplete expression`, `Malformed expression`, `Division by zero`, `Invalid token`, or `Invalid operand`.

Constraints:

- $1 \leq q \leq 1000$
- Operands are integers in the range -10^6 to 10^6
- Number of tokens in a single expression ≤ 100
- Operators are limited to $+$, $-$, $*$, $/$, $\%$ (binary, two operands each); division and modulo truncate toward zero
- Stack capacity, if array-based, is fixed at compile time (e.g., 100)

Error Cases:

- **Incomplete expression** – raised when an operator is encountered but the stack does not hold the two operands required to evaluate it (e.g., in `"5 +"`, only one operand is available before the operator).
- **Malformed expression** – raised when, after every token has been processed, more than one value remains on the stack, meaning some operands were never combined by an operator (e.g., `"5 3 + 8"` leaves two values on the stack).
- **Division by zero** – raised via `isValid()` on the constructed `DivideOperator` or `ModuloOperator` when the divisor evaluates to zero (e.g., `"10 0 /"` attempts to divide 10 by 0).
- **Invalid token** – raised when a token is neither a valid integer nor one of the supported operators (e.g., `"5 3 @ +"` contains the unrecognized token `"@"`).
- **Invalid operand** – raised when a token is a syntactically valid number but is not an integer, since only integer operands are supported (e.g., `"7.5 3 +"` attempts to use the non-integer operand 7.5).

Example 1:**Input:**

```
10
45 30 + 2 * 15 3 / -
10 0 /
5 +
5 3 + 8
7 3 %
5 3 @ +
7.5 3 +
7 3 / 2 %
-5 3 +
5 --3 +
```

Output:

```
145
Division by zero
Incomplete expression
Malformed expression
1
Invalid token
Invalid operand
0
-2
Invalid token
```

Q4. Infix to Postfix Using Stack ADT**Question:**

Design a **Stack ADT** using a singly linked list and use it to convert a given infix mathematical expression into its equivalent postfix expression.

The expression consists of non-negative integers, arithmetic operators, and parentheses. The operators supported are $+$, $-$, $*$, $/$, and $^$.

The operator precedence is as follows:

Operator	Precedence	Associativity
$^$	3	Right-to-left
$*$, $/$	2	Left-to-right
$+$, $-$	1	Left-to-right

Parentheses must be handled according to the standard rules of arithmetic expressions.

Mandatory Requirement:

You **cannot** use any built-in STL stack or other STL container to implement the stack. You must design and implement your own **Stack ADT** using a singly linked list with classes/structs and pointers.

The Stack ADT must provide at least the following operations:

- **push(x)** – Insert an element at the top of the stack.
- **pop()** – Remove and return the top element.
- **peek()** – Return the top element without removing it.
- **isEmpty()** – Check whether the stack is empty.

The linked-list node should contain the data and a pointer to the next node.

Use private data members in Stack to demonstrate encapsulation.

Properly manage dynamically allocated memory.

Include an appropriate constructor and destructor.

Handle the case where **pop()** or **peek()** is called on an empty stack appropriately.

Expected Input:

The first line contains an integer T , representing the number of test cases.

Each of the next T lines contains a valid infix expression. All tokens in the expression are separated by spaces.

Expected Output:

For each test case, print the equivalent postfix expression. Each operand and operator in the postfix expression must be separated by a single space.

Constraints:

- $1 \leq T \leq 100$
- $1 \leq \text{length of each expression} \leq 10^4$
- Each operand is a non-negative integer.
- Each expression is guaranteed to be valid.
- Tokens in the expression are separated by spaces.
- The algorithm should run in $O(N)$ time for each expression.

- The stack must be implemented using a singly linked list.
- No STL stack or other STL container may be used to implement the stack.

Example 1:

Input:

```
5
2 + 3 * 4
( 2 + 3 ) * 4
10 + 2 * 6
100 * ( 2 + 12 )
2 ^ 3 ^ 2
```

Output:

```
2 3 4 * +
2 3 + 4 *
10 2 6 * +
100 2 12 + *
2 3 2 ^ ^
```

Example 2:

Input:

```
3
12 + 34 * 5
( 10 + 20 ) / ( 5 - 2 )
8 * 3 + 6 / 2
```

Output:

```
12 34 5 * +
10 20 + 5 2 - /
8 3 * 6 2 / +
```

Q5. Infix to Prefix Conversion Using Stack ADT

A spacecraft navigation system receives mathematical expressions from the ground station in infix notation, but its onboard processor can execute only prefix expressions. You are required to develop an object-oriented module in C++ that converts the received expressions into prefix notation before they are processed.

Design and implement an object-oriented program to convert a given infix mathematical expression into its equivalent prefix expression.

The solution must use a custom **Stack ADT implemented using a singly linked list**. The use of STL `stack` or any other STL container for implementing the stack is not permitted.

Solution Requirements:

- Create a `Node` structure containing the data and a pointer to the next node.
- Implement a `Stack` class using the singly linked list. Its data members must be private and it must provide `push()`, `pop()`, `peek()`, and `isEmpty()` operations.
- Create an abstract class `ExpressionConverter` with a pure virtual function `convert()`.
- Create a `PrefixConverter` class derived from `ExpressionConverter` and override `convert()` to perform the infix-to-prefix conversion using the custom Stack ADT.
- Demonstrate encapsulation, inheritance, polymorphism, constructors, destructors, and proper dynamic memory management.
- Handle `pop()` and `peek()` appropriately when the stack is empty.

Expression Details:

The expression contains non-negative integers, parentheses, and the operators `+`, `-`, `*`, `/`, and `^`.

Operator	Precedence	Associativity
<code>^</code>	3	Right-to-left
<code>*</code> , <code>/</code>	2	Left-to-right
<code>+</code> , <code>-</code>	1	Left-to-right

The conversion must correctly handle precedence, associativity, and nested parentheses. The exponentiation operator must remain right-associative.

Input:

The first line contains an integer T . Each of the next T lines contains a valid infix expression. All tokens are separated by spaces.

Output:

For each test case, print the equivalent prefix expression with a single space between consecutive tokens.

Constraints:

- $1 \leq T \leq 100$
- Expression length $\leq 10^4$
- Each operand is a non-negative integer.
- Each expression is guaranteed to be valid.
- The conversion should run in $O(N)$ time per expression.

Example:

Input:

```
4
2 + 3 * 4
( 2 + 3 ) * 4
10 + 2 * 6
2 ^ 3 ^ 2
```

Output:

```
+ 2 * 3 4
* + 2 3 4
+ 10 * 2 6
^ 2 ^ 3 2
```

Q6. Prefix Expression Evaluation Using Stack ADT**Question:**

Design a **Stack ADT** using an array and use it to evaluate a given valid prefix mathematical expression.

The expression consists of integers and the arithmetic operators $+$, $-$, $*$, and $/$.

The prefix expression must be evaluated by scanning the expression from **left to right**.

Mandatory Requirement:

You **cannot** use any built-in STL stack or other STL container to implement the stack. You must design and implement your own **Stack ADT** using an array.

The Stack ADT must follow the **LIFO (Last In, First Out)** principle and provide at least the following operations:

- `push(x)` – Insert an element at the top of the stack.
- `pop()` – Remove and return the top element.
- `peek()` – Return the top element without removing it.
- `isEmpty()` – Check whether the stack is empty.

The stack must be implemented using an array with an appropriate `top` variable to keep track of the top element.

Use private data members in **Stack** to demonstrate encapsulation.

Properly manage the dynamically allocated array memory.

Include an appropriate constructor and destructor.

Handle the case where `pop()` or `peek()` is called on an empty stack appropriately.

Expected Input:

The first line contains an integer T , representing the number of test cases.

For each test case, the first line contains an integer N , representing the number of tokens in the prefix expression.

The next line contains N space-separated tokens representing the prefix expression.

Expected Output:

For each test case, print the evaluated result of the prefix expression on a separate line.

Constraints:

- $1 \leq T \leq 10$
- $1 \leq N \leq 10^5$
- Each operand is an integer in the range $[-10^9, 10^9]$.
- Each expression is guaranteed to be valid.
- Division by zero will not occur.
- Intermediate results fit in a 64-bit integer.
- The stack must be implemented using an array.
- No STL stack or other STL container may be used to implement the stack.
- Each stack operation must run in $O(1)$ time.
- The overall evaluation must run in $O(N)$ time for each expression.
- Auxiliary space must be $O(N)$.

Example 1:

Input:

```
2
5
+ 5 3
7
* + 2 3 - 8 4
```

Output:

```
8
20
```

Explanation:

For the first test case:

$$+ 5 3 = 5 + 3 = 8$$

For the second test case:

$$\begin{aligned} &* + 2 3 - 8 4 \\ &= (2 + 3) * (8 - 4) \\ &= 5 * 4 = 20 \end{aligned}$$

Example 2:

Input:

```
3
3
- 10 4
5
/ * 8 3 2
7
+ * 2 5 / 20 4
```

Output:

```
6
12
14
```